Experience-Based Testing

Experience based technique

- Experience-based Tests / Intuitive Testing
- Experience-based tests are based on the tester's
  - Skills and intuition
  - Experience with similar applications
  - Experience with similar technologies
- Rather than being pre-designed, experience-based tests are often created during test execution (i.e. test strategy is dynamic/on-the-fly)
- Only document if there's defect
- Tests are frequently "time-boxed" (i.e. brief periods of testing focused on specific test conditions)

Experience based technique (cont.)

- Test knows typical problems.
- Focus on:
  - Typical faults
  - Unclear items
  - Special values
  - Special connections and contexts
  - Risk factors
  - Use of the program and its environment
- Very dependent on knowledge, skill and experience!
- Good addition to black and white box!

Experience based Types



Error Guessing: Characteristics

- A test design technique, where the **experience and knowledge of the tester is used to anticipate what defects** might be present in the component or system under test as a result of errors made and to design tests specifically to expose them
- Method for choosing test cases, based on knowledge and experience of the tester
- Structured approach
  - Create a **defect catalog** containing as many defects and effects of defects as possible
  - **Design test cases** which target those defects

© Copyright 2011 to MSTB/ GTB V 1.0

Error Guessing: Procedure(1)

- Possibly the most often used test design technique (often also called intuitive test design) Tests are derived from the capability and the intuition of the tester and out of his/her experience with similar applications and technologies
- Error Guessing can be used to support systematic test design techniques
- Determines tests, which are not/or only with difficulty designed by systematic test design techniques
- Attention: Error guessing can achieve very different degrees of efficiency, as they are dependent on the experience of the tester

© Copyright 2011 to MSTB/ GTB V 1.0

Error Guessing: Procedure (2)

- Structured Approach
  - Create a list of possible defects and design the test cases, which target those defects
  - This list of defective states and effects can be created on the basis of experience, available data and general knowledge relating to software defects
  - The list can be also of great benefit to users, who can be made aware of possible problems and difficulties before implementation. This can be useful for defect avoidance.

© Copyright 2011 to MSTB/ GTB V 1.0

Error Guessing: Catalog of Defects

- List with possible defects and error-prone situations
  - Extensive experience often only in the heads of experienced testers
  - Experience of frequently occurring defects are recorded and are available for all testers
- Problems:
  - Timeliness and relevance
  - Defect catalogs are not domain- or product-specific
- General defect catalogs should be adopted
- Definition and care of own defect catalog is always needed

© Copyright 2011 to MSTB/ GTB V 1.0

Error Guessing: Catalog of Defects

- Numerous published defect catalogues Cem Kaner, Jack Falk, & Hung Quoc Nguyen, Testing Computer Software contains a list of above 400 defects
- James A. Whittaker: How to Break Software: A Practical Guide to Testing contains possible „Attacks“, which could lead to a defect
- James A. Whittaker: How to Break Software Security: contains possible „Security Attacks“
- Mike Andrews and James A. Whittaker: How to Break Web Software: Functional and Security Testing of Web Applications and Web Services
- Greg Hoglund and Gary McGraw: Exploiting Software: How to Break Code

© Copyright 2011 to MSTB/ GTB V 1.0

Exploratory Testing: Learn, Test and Execute Simultaneously

- Informal testing, in which no test preparation takes place and no specific test design technique is used
- No expected results are specified and the test run happens more or less arbitrarily
- Test case analysis and test case design, test implementation and execution, test recording and especially also the learning process all take place at the same time
- Basis is a test charter, from which the test goals and possible test ideas can be taken
- Execution in a fixed time frame
- Especially adequate if there are only few or unqualified specification, when testing under time pressure, or if other formal test design techniques are to be complemented

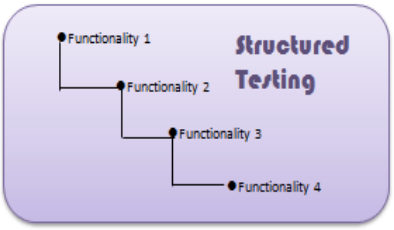
© Copyright 2011 to MSTB/ GTB V 1.0

Exploratory Testing

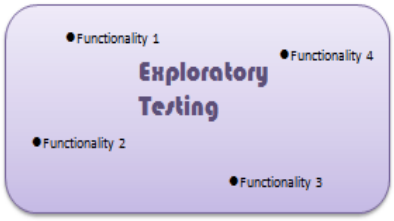


- All happen at the same time
- Useful when there is severe time pressure or inadequate specifications

Exploratory vs. Error Guessing



Functionalities are checked in a structured manner



Functionalities are checked in a ad-hoc manner

Exploratory vs. Structured

Structured Testing	Exploratory Testing
Directed from requirements	Directed from requirements and exploring during testing
Determination of test cases well in advance	Determination of test cases during testing
Confirmation of testing with the requirements	Investigation of system or application
Emphasizes on prediction and decision making	Emphasizes on adaptability and learning
Involves confirmed testing	Involves Investigation
Is about Controlling tests	Is about Improvement of test design
Like making a speech - you read from a draft	Like making a conversion - its spontaneous
The script is in control	The tester's mind is in control

Exploratory Testing: Qualities and Activities

- Qualities
  - Parallel test design and test execution. Development of strategies to scrutinize the product, the definition and the execution of test cases
  - Verifiable results. Results and experiences of previous tests will be documented, evaluated and may influence the next tests
  - Exploration : "Getting to know" the product (Functions used, data, immature product areas, …)
  - Heuristics: Guidelines and rules of thumb simplify the choice of test cases
- Activities Test Preparation
  - Test Execution
  - Documentation
  - Test Evaluation

Exploratory Testing: Test Preparation

Unit

Closed part of the product

Test duration 1-2 days

Working Units

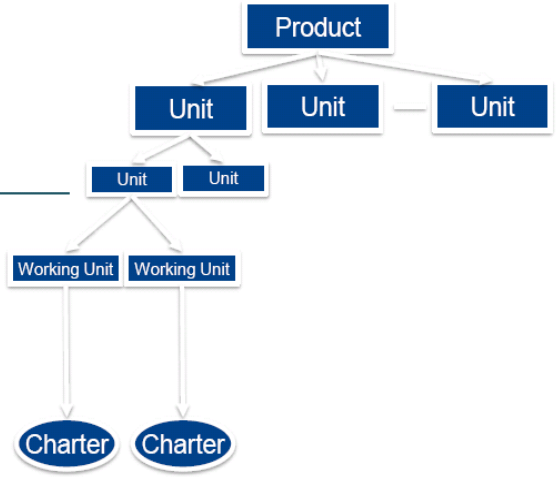
To be worked off in a test session „in one go“

Test duration 1-2 hours

Charter:

Provide goal and tasks

for the test session



Nach: Rapid Software Testing, copyright © 1996-2002 James Bach

Exploratory Testing: The 5 stages

Create Bug Taxonomy
- Categorize common defects found in past projects
- Analyze the root cause of the faults
- Find the risk and develop ideas to test the application.

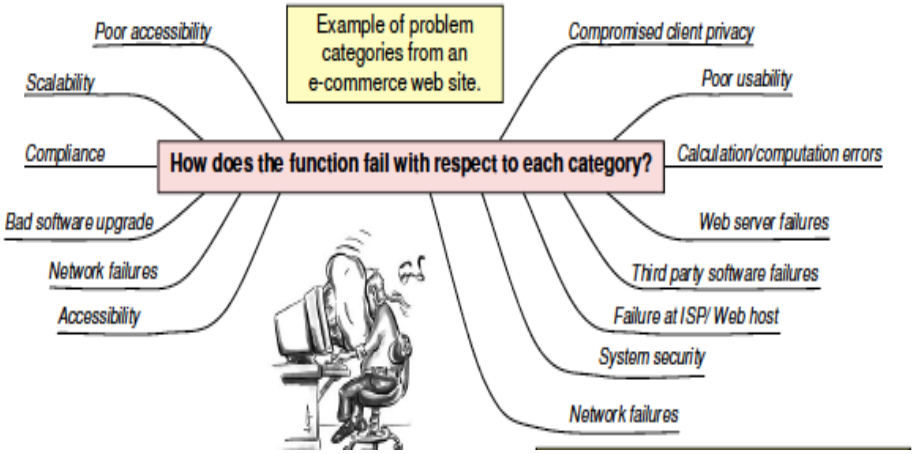
Test Charter
- Suggest
  - What to test
  - How it can be tested
  - What needs to be looked

Time Box
- Includes pair of tester working together not less than 90 minutes.
- There should not be any interrupted time in those 90 minutes session.
- Time box can be extended or reduced by 45 minutes.

Review Results
- Evaluation of the defects
- Learning from the testing
- Analysis of coverage areas.

Debriefing
- Compilation of the output results
- Compare the results with the charter
- Check whether any additional testing is needed.

Bug Taxonomy Example



Test charter template example

Test Charter id and unique name

Actor/Theme:	<The user role you as a tester is going to play when using the system or theme when testing system characteristics>
Purpose:	<The purpose of the charter including what particular objectives the actor is supposed to achieve or what the expected outcome of the theme exploration should be>
Setup:	<What needs to be in place in order to start using the intended function or characteristic , i.e. the precondition to be able to test (configuration and the initial system state, e.g. a start menu should appear on the screen with login fields or what background load scenarios should be up and running)>
Priority:	<High/low, based on what the priority of the requirements are, or other considerations (risk level)>
Reference:	<Specification (e.g. Requirements Specification), risk or other information source>
Data:	<Reference to a data file with input data to be used, e.g. an Excel sheet. Variations of data to be made based on the initial data can also be specified, e.g. use boundary values of valid and invalid input data for all parameters, use pairings of input data to catch combinatorial problems>
Activities:	<What the actor should do with the system, e.g. Log on to the system as a super user: - Add a new user. - Attempt to delete a non existing user>
Oracle notes:	<Things to pay special attention to regarding system response and what is expected as an adequate/correct outcome, e.g. Check that correct error messages are given that informs the user about what mistake he made and how to do it correctly next time. It could also be a reminder to the tester to check for buffer consumption or other important resource usage>
Variations:	<Additional system or user events that could occur at anytime the normal activities are performed as described above under Activities, e.g. another super user logs on to the system trying to manipulate the same data as the first is working on, or the network connection suddenly fails>

Test ideas

Choosing test Techniques



- How do you chose the right technique?
  - Type of system
  - Standards
  - Customer or contractual requirements
  - Level of risk
  - Type of risk
  - Testing objectives
  - Documentation available
  - Knowledge / skills of the testers
  - Time and budget
  - Development processes
- Pick the right techniques for the right situation

Factors in Choosing Techniques

1. **Test objectives** - If the test objective is simply to gain confidence that the soft ware will cope with typical operational tasks then use cases would be a sensible approach. If the objective is for very thorough testing then more rigorous and detailed techniques (including structure-based techniques) should be chosen.

2. **Level and type of risk** - The greater the risk (e.g. safety-critical systems), the greater the need for more thorough and more formal testing. Commercial risk may be influenced by quality issues (so more thorough testing would be appropriate) or by time-to-market issues (so exploratory testing would be a more appropriate choice)

Factors in Choosing Techniques (cont.)

3. **Type of system** - the type of system (e.g. embedded, graphical, financial, etc.) will influence the choice of techniques. For example, a financial application involving many calculations would benefit from boundary value analysis.
4. **Regulatory standard** - Some industries have regulatory standards or guidelines that govern the testing techniques used. For example, the aircraft industry requires the use of equivalence partitioning, boundary value analysis and state transition testing for high integrity systems together with statement, decision or modified condition decision coverage depending on the level of software integrity required.

Factors in Choosing Techniques (cont.)

5. **Customer or contractual requirements** - Sometimes contracts specify particular testing techniques to use (most commonly statement or branch coverage).

Summary

- We learned about
  - Identifying Test Conditions
  - Designing Test Cases from the Test Conditions
  - Creating Test Procedure Specifications to sequence and schedule our Test Cases
  - The importance of traceability to requirements and specification of expected results
- We learned about the difference between Black and White Box testing
  - White Box (Structure-based) Testing is based upon the structure of the program code
  - Black Box (Specification Based) Testing is without reference the internal working of the program code
  - The reasons why both are useful

Summary

- We learned about the different Black box techniques, mainly:
  - Equivalence Partitioning
  - Boundary Value Analysis
  - State Transition Testing
  - Decision Tables
  - Use Case Testing
  - Experience based Testing
- We learned about the different White box techniques, mainly:
  - Statement Testing
  - Decision Testing
- For all Black and White box techniques we learned why they are of use and for which test levels they are typically applied